Prescribing Security: Uncovering 5 CVEs in a Healthcare Application

seccore.at/blog/cves-meona

Benjamin Floriani, Patrick Pongratz May 13, 2026

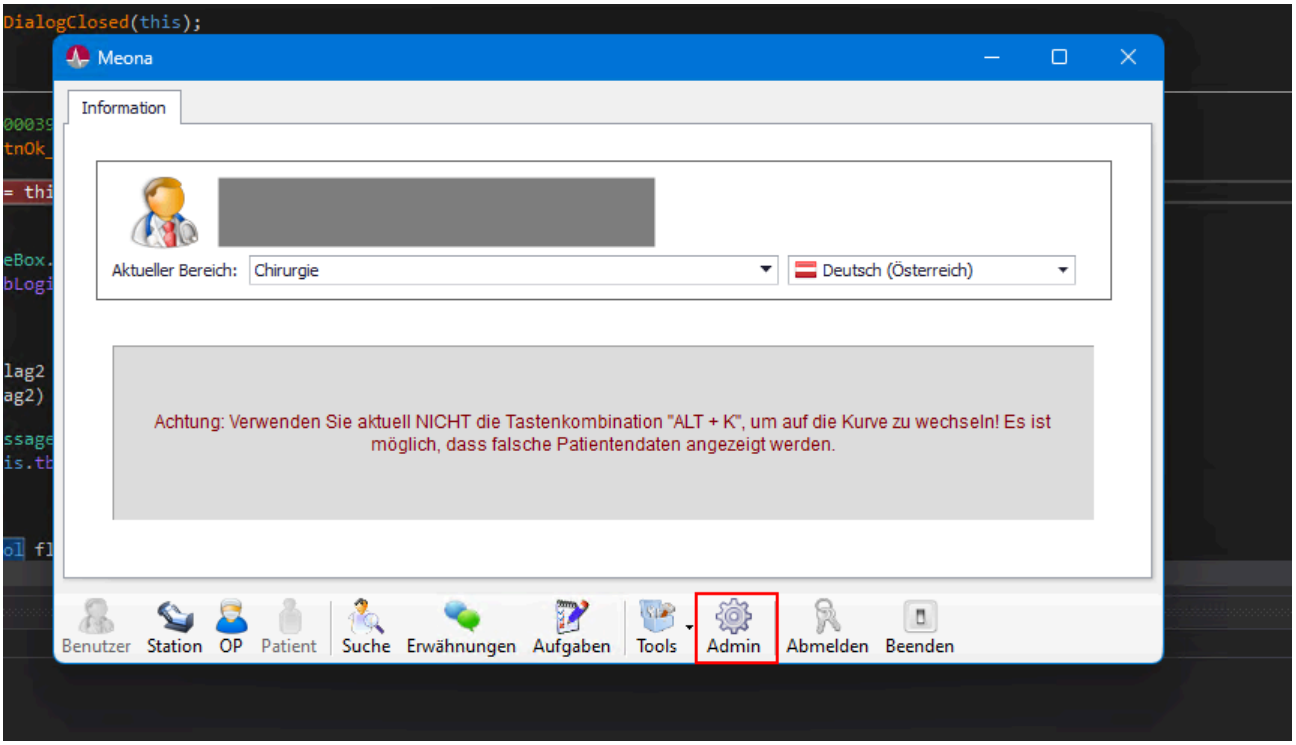
Note: This is not the final version of this blog post. Some technical details are being withheld until the final patches for all vulnerabilities have been released.

Before diving into the found vulnerabilities, we want to give a huge shoutout to the Austrian **CERT**[1]. The CERT supported us throughout the whole coordinated vulnerability process (CVD), handled the communication with the ENISA for us and gave us regular insights and information about the current status. If anybody thinks about going through a CVD process, we would highly recommend doing it with the CERT.

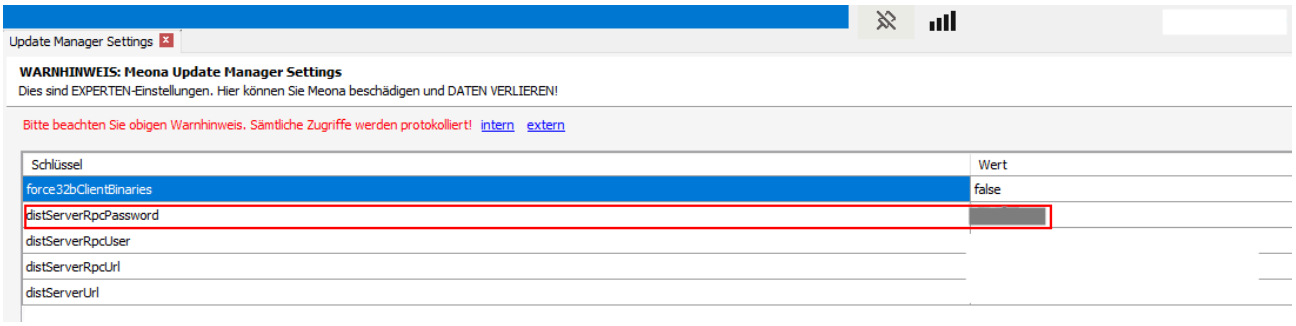
The found vulnerabilities in this blog post affect the application **Meona** from **Mesalvo**[2], which we found during a recent red team engagement. The affected version of Meona was **2025.04 5+323020** and the version of the launcher was **19.06.2020 15:11:49** (yes, just a timestamp). The application is used in the healthcare sector to manage patients, and is built on a client and server architecture. The client communicates through HTTP with the backend server. During our red team engagement, we abused the application for lateral movement, mail spoofing and password leakage. Below you can find each CVE and how we abused it.

Client-Side Authorization (CVE-2026-0856)

To start off, we first needed to find a way into the application, since we only had regular credentials without admin access to Meona. Examining the architecture of the .NET application and tinkering with it, we found out that the backend server did not verify the permissions of the given credentials. Because of that, we were able to access the admin panel with normal user credentials:

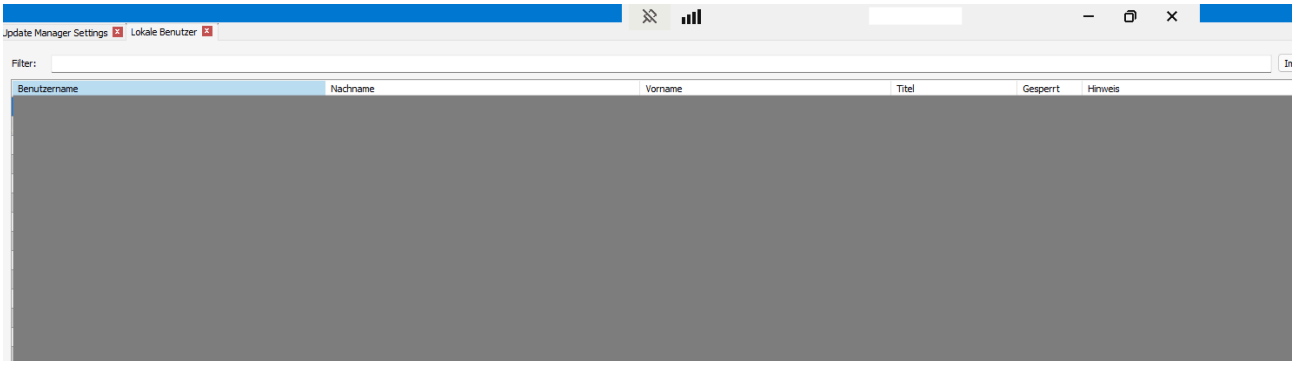


This access provides full control over all the data and functions inside the admin portal:

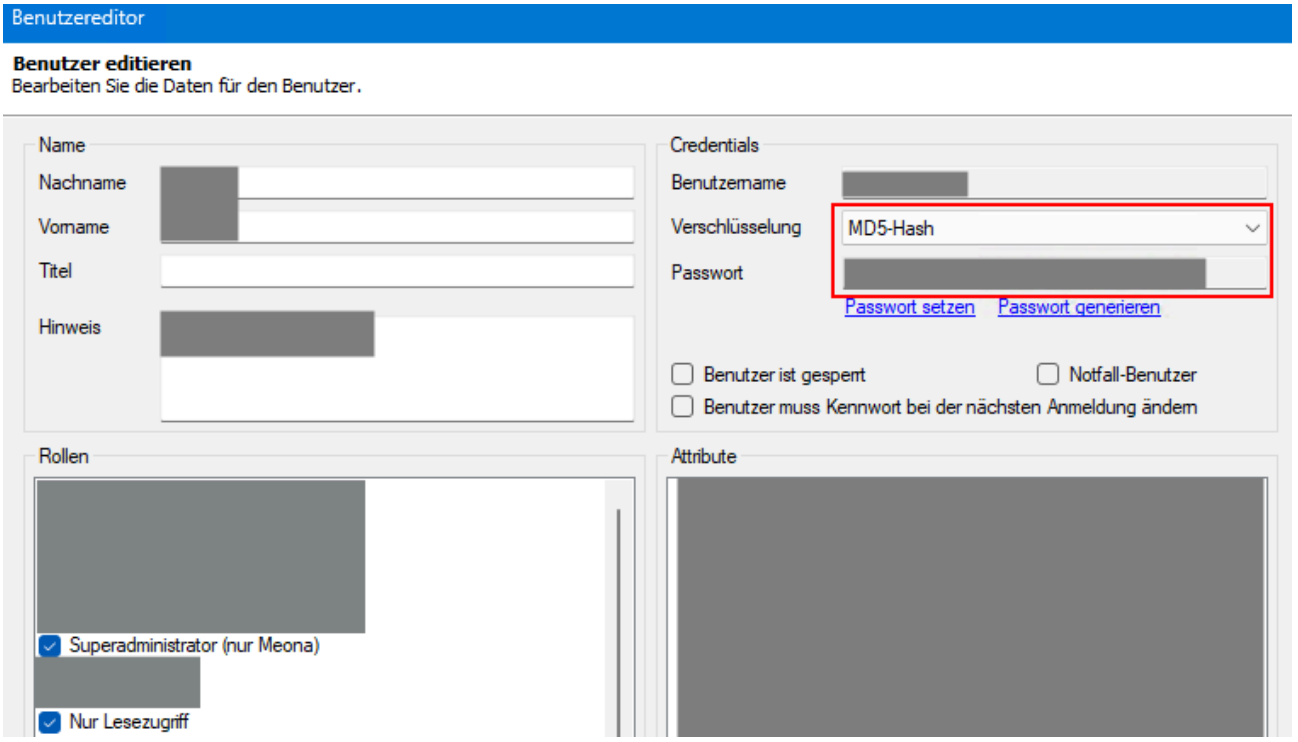


Plain Text Storage of Passwords (CVE-2026-0857)

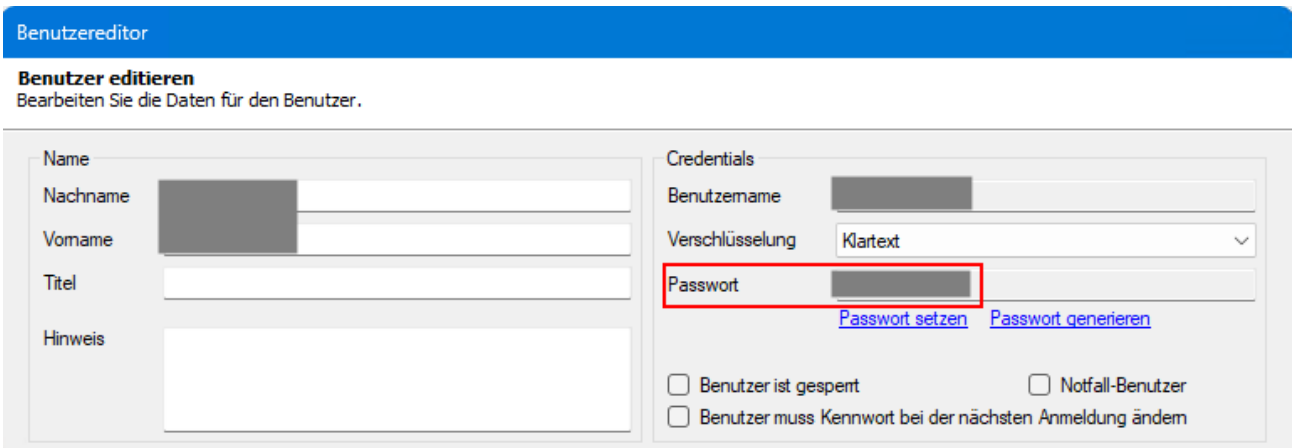
After getting access to the admin panel, the users of the application could be examined:



Browsing through the users, several super admin accounts were found, with their respective passwords hashed using the MD5 algorithm:



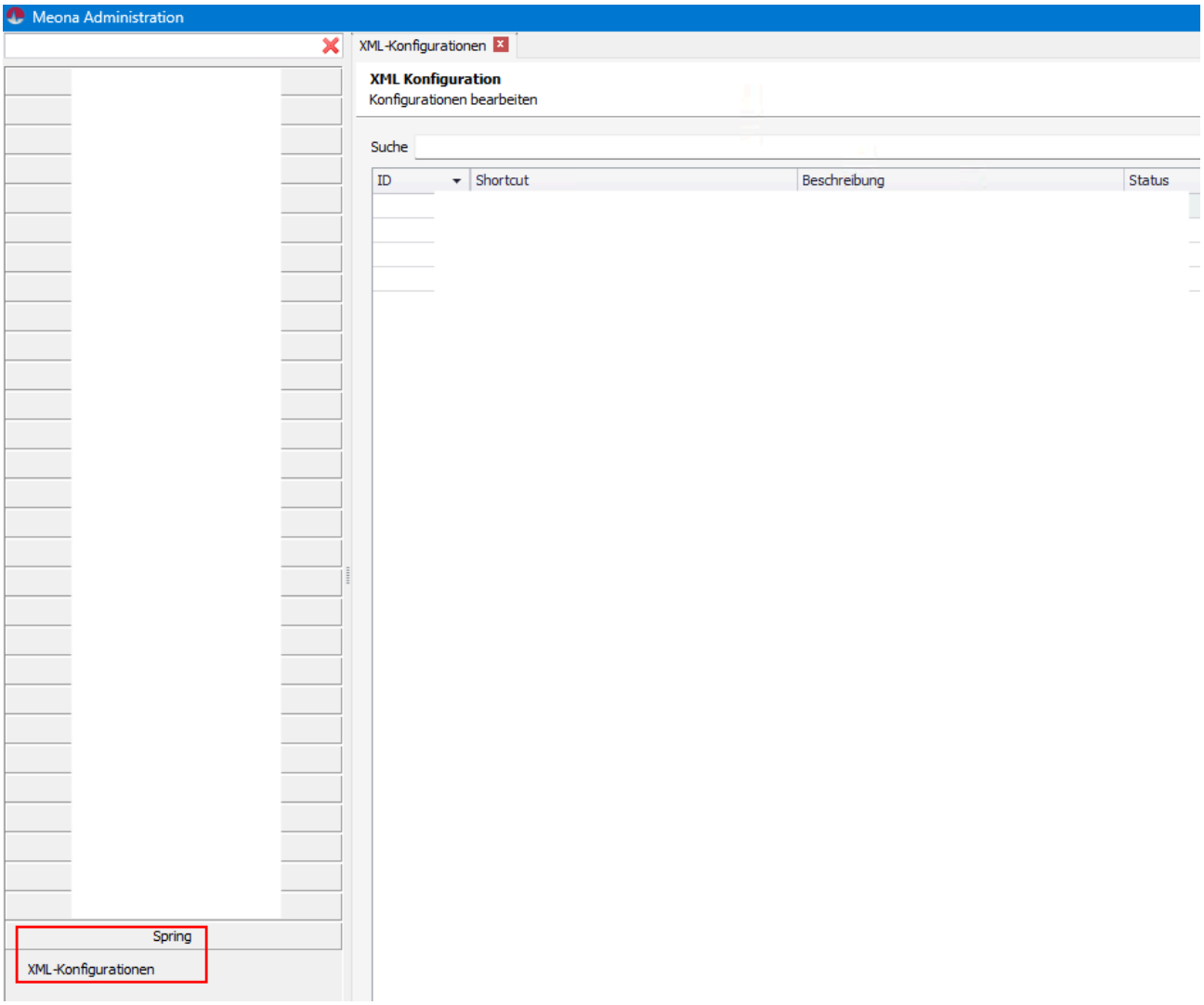
Even more critical was the fact that some passwords were also stored in plaintext:



These hashes were cracked in a short amount of time and were used in password spraying attacks together with the found plaintext passwords. We used this for lateral movement through the network through additional compromised accounts.

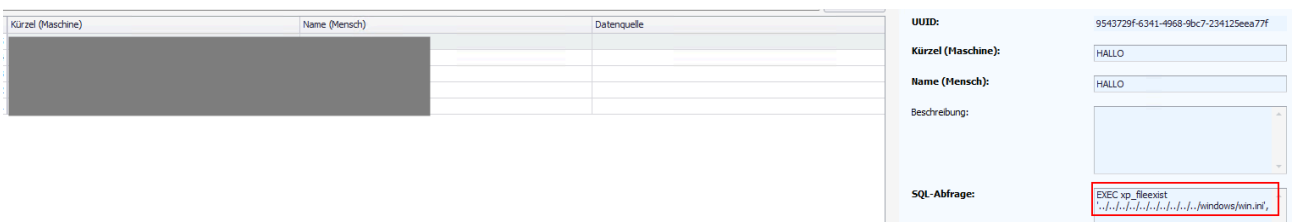
Code Execution (CVE-2026-22314)

Within the application, we found another way to laterally move through the network by abusing different administrative features. Scripts could be added in different programming languages or XML-based configurations could be changed to execute arbitrary code on each client Meona is running on:



Accessible SQL Interface (CVE-2026-22315)

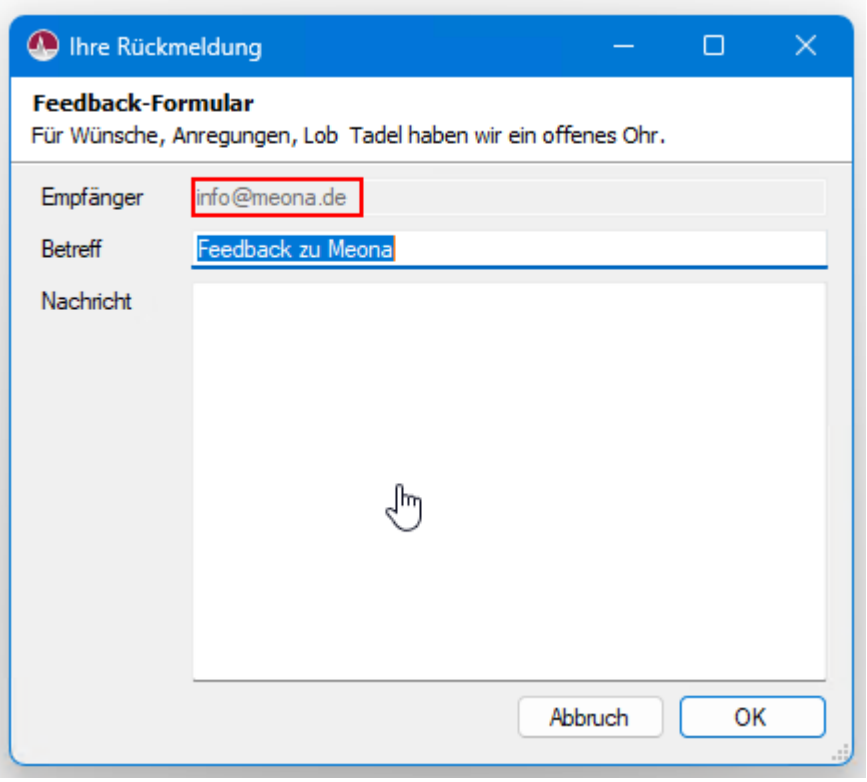
Another feature of the application allowed [NTLM relay attacks] through an accessible SQL interface, since common SQL stored procedures were active:



Among other things, also the (cleartext) passwords of all users could be read out through the same SQL interface, bypassing any logging mechanisms.

Recipient Email Address Manipulation (CVE-2026-25602)

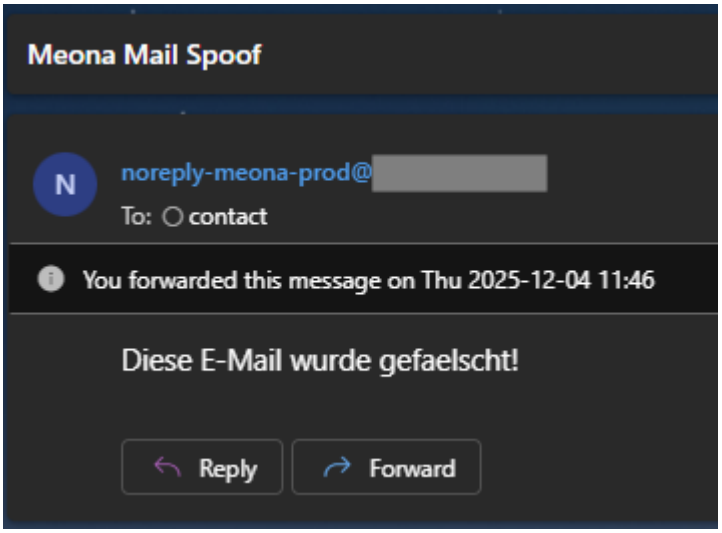
The last thing we abused within our red team engagement was the feedback report functionality. As it can be seen in the screenshot, the mail address was hardcoded into the client:



To change this, either the string within the client could be manipulated, or (the funnier part) the traffic could be analyzed:

```
POST /medication/kernel HTTP/1.1\r\n
[redacted]\r\n\r\n
Accept-Encoding: gzip, deflate, br\r\n\r\n
X-Meona-Profile: [redacted]\r\n\r\n
X-Meona-Client: [redacted]\r\n\r\n
Accept-Language: de-AT\r\n\r\n
Content-Type: application/x-hessian\r\n\r\n
Host: [redacted]\r\n\r\n
Content-Length: 108\r\n\r\n
Connection: keep-alive\r\n\r\n
c\0\1m\0\0bsendMail__3Vt\0\07[stringl\0\0\0\01S\0\12contact@seccore.atzS\0\10Meona Mail SpoofS\0\1eDiese E-Mail wurde gefaelscht!z
```

As it can be seen, the application uses serialized Hessian[3] objects for communication. Once the protocol was understood, the mail could simply be swapped out to an arbitrary address, together with the content. Since the server component does not verify the recipient address, it accepts the change and the message arrived at the new destination, sent from a legitimate mail server:



Since the mail was sent from an internal address (noreply-meona-prod@customer.at), this feature was abused for internal social engineering.

Timetable

During the CVD we had the following timeline of relevant events:

Date	Description
2026-01-08	Requested CVE identifiers and notified manufacturer
2026-01-12	Received CVE identifiers
2026-01-26	Manufacturer contacted SecCore
2026-01-27	Explained vulnerabilities to manufacturer
2026-02-19	Asked manufacturer for an update
2026-02-25	Asked manufacturer for an update
2026-04-15	Meeting with manufacturer, re-explained the vulnerabilities
2026-05-04	Manufacturer wants to apply patches in May / June
2026-05-04	Manufacturer requested publication of the CVEs
2026-05-13	Initial blog post release